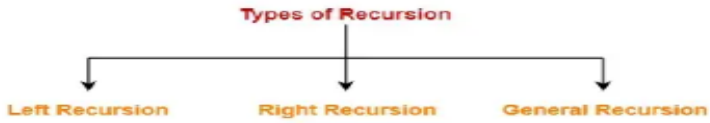
Left-Recursion



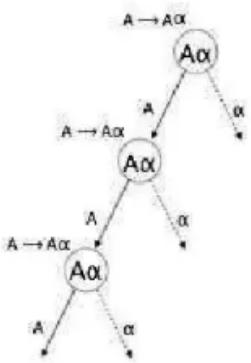
1. Left-Recursion $S \rightarrow Sa / \in$
2. Right-Recursion $S \rightarrow aS / \in$
3. General-Recursion $S \rightarrow aSb / \in$

A production of grammar is said to have left recursion if the leftmost variable of its RHS is same as variable of its LHS.

A grammar containing a production having left recursion is called as Left Recursive Grammar.

Left-Recursion

$A \rightarrow A\alpha / \beta$



Left recursion is considered to be a problematic situation for Top down parsers.

Therefore, left recursion has to be eliminated from the grammar.

Left-Recursion Elimination

we can eliminate left recursion by replacing the pair of productions with-

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \in$

Problem-01:

Consider the following grammar and eliminate left recursion-

$A \rightarrow ABd / Aa / a$

$B \rightarrow Be / b$

Solution-

The grammar after eliminating left recursion is-

$A \rightarrow aA'$

$A' \rightarrow BdA' / aA' / \in$

$B \rightarrow bB'$

$B' \rightarrow eB' / \in$

Problem-02:

Consider the following grammar and eliminate left recursion-

$E \rightarrow E + E / E \times E / a$

Solution-

The grammar after eliminating left recursion is-

$E \rightarrow aA$

$A \rightarrow +EA / xEA / \in$

Problem-03:

Consider the following grammar and eliminate left recursion-

$E \rightarrow E + T / T$

$T \rightarrow T \times F / F$

$F \rightarrow id$

Solution-

The grammar after eliminating left recursion is-

$E \rightarrow TE'$

$E' \rightarrow +TE' / \in$

$T \rightarrow FT'$

$T' \rightarrow xFT' / \in$

$F \rightarrow id$

Problem-04:

$S \rightarrow (L) / a$

$L \rightarrow L , S / S$

Solution-

The grammar after eliminating left recursion is-

$S \rightarrow (L) / a$

$L \rightarrow SL'$

$L' \rightarrow ,SL' / \in$

Consider the following grammar and eliminate left recursion-

$S \rightarrow S0S1S / 01$

Solution-

The grammar after eliminating left recursion is-

$S \rightarrow 01A$

$A \rightarrow 0S1SA / \in$

Consider the following grammar and eliminate left recursion-

$S \rightarrow A$

$A \rightarrow Ad / Ae / aB / ac$

$B \rightarrow bBc / f$

Solution-

$S \rightarrow A$

$A \rightarrow aBA' / acA'$

$A' \rightarrow dA' / eA' / \in$

$B \rightarrow bBc / f$

A → Ba / Aa / c

B → Bb / Ab / d

Step-01:

First let us eliminate left recursion from A → Ba / Aa / c

A → BaA' / cA'

A' → aA' / ∈

Now, given grammar becomes-

A → BaA' / cA'

A' → aA' / ∈

B → Bb / Ab / d

Step-02:

Substituting the productions of A in B → Ab, we get the following grammar-

A → BaA' / cA'

A' → aA' / ∈

B → Bb / BaA'b / cA'b / d

Step-03:

Now, eliminating left recursion from the productions of B, we get the following grammar-

A → BaA' / cA'

A' → aA' / ∈

B → cA'bB' / dB'

B' → bB' / aA'bB' / ∈

FIRST and FOLLOW

First and Follow sets are needed so that the parser can properly apply the needed production rule at the correct position.

FIRST

First(α) is a set of terminal symbols that begin in strings derived from α.

Consider the production rule-

A → abc / def / ghi

Then, we have- First(A) = { a , d , g }

FIRST and FOLLOW

FIRST

For a production rule $X \rightarrow \in$,

First(X) = { $\in$ }

For any terminal symbol 'a',

First(a) = { a }

FIRST and FOLLOW

FOLLOW

Follow(α) is a set of terminal symbols that appear immediately to the right of α.

- For the start symbol S, place $ in Follow(S).
- For any production rule $A \rightarrow \alpha B$,

  Follow(B) = Follow(A)
- For any production rule $A \rightarrow \alpha B \beta$,

  If $\in \notin$ First(β), then Follow(B) = First(β)

  If $\in \in$ First(β), then Follow(B) = { First(β) – $\in$ } ∪ Follow(A)

FIRST and FOLLOW

$\in$ may appear in the first function of a non-terminal.

$\in$ will never appear in the follow function of a non-terminal.

Before calculating the first and follow functions, eliminate Left Recursion from the grammar, if present.

PRACTICE PROBLEMS BASED ON CALCULATING FIRST AND FOLLOW-

Problem-01:

Calculate the first and follow functions for the given grammar-

$$S \rightarrow aBDh$$
$$B \rightarrow cC$$
$$C \rightarrow bC / \epsilon$$
$$D \rightarrow EF$$
$$E \rightarrow g / \epsilon$$
$$F \rightarrow f / \epsilon$$

Solution-

The first and follow functions are as follows-

First Functions-

- First(S) = { a }
- First(B) = { c }
- First(C) = { b , ∈ }
- First(D) = { First(E) – ∈ } ∪ First(F) = { g , f , ∈ }
- First(E) = { g , ∈ }
- First(F) = { f , ∈ }

Follow Functions-

- Follow(S) = { $ }
- Follow(B) = { First(D) – ∈ } ∪ First(h) = { g , f , h }
- Follow(C) = Follow(B) = { g , f , h }
- Follow(D) = First(h) = { h }
- Follow(E) = { First(F) – ∈ } ∪ Follow(D) = { f , h }

- Follow(F) = Follow(D) = { h }

Problem-02:

Calculate the first and follow functions for the given grammar-

$$S \rightarrow A$$
$$A \rightarrow aB / Ad$$
$$B \rightarrow b$$
$$C \rightarrow g$$

Solution-

We have-

- The given grammar is left recursive.
- So, we first remove left recursion from the given grammar.

After eliminating left recursion, we get the following grammar-

$$S \rightarrow A$$
$$A \rightarrow aBA'$$
$$A' \rightarrow dA' / \epsilon$$
$$B \rightarrow b$$
$$C \rightarrow g$$

Now, the first and follow functions are as follows-

First Functions-

- First(S) = First(A) = { a }
- First(A) = { a }
- First(A') = { d , ∈ }
- First(B) = { b }
- First(C) = { g }

Follow Functions-

- Follow(S) = { $ }
- Follow(A) = Follow(S) = { $ }
- Follow(A') = Follow(A) = { $ }
- Follow(B) = { First(A') – ∈ } ∪ Follow(A) = { d , $ }
- Follow(C) = NA

Problem-03:

Calculate the first and follow functions for the given grammar-

$$S \rightarrow (L) / a$$

$$L \rightarrow SL'$$

$$L' \rightarrow ,SL' / \in$$

Solution-

The first and follow functions are as follows-

First Functions-

- First(S) = { (, a }
- First(L) = First(S) = { (, a }
- First(L') = { , , ∈ }

Follow Functions-

- Follow(S) = { $ } ∪ { First(L') – ∈ } ∪ Follow(L) ∪ Follow(L') = { $, , ,) }
- Follow(L) = {) }
- Follow(L') = Follow(L) = {) }

Problem-04:

Calculate the first and follow functions for the given grammar-

$$S \rightarrow AaAb / BbBa$$

$$A \rightarrow \in$$

$$B \rightarrow \in$$

Solution-

The first and follow functions are as follows-

First Functions-

- First(S) = { First(A) – ∈ } ∪ First(a) ∪ { First(B) – ∈ } ∪ First(b) = { a , b }
- First(A) = { ∈ }
- First(B) = { ∈ }

Follow Functions-

- Follow(S) = { $ }
- Follow(A) = First(a) ∪ First(b) = { a , b }
- Follow(B) = First(b) ∪ First(a) = { a , b }

Problem-05:

Calculate the first and follow functions for the given grammar-

$$E \rightarrow E + T / T$$

$$T \rightarrow T \times F / F$$

$$F \rightarrow (E) / id$$

Solution-

We have-

- The given grammar is left recursive.
- So, we first remove left recursion from the given grammar.

After eliminating left recursion, we get the following grammar-

---

$$E \rightarrow TE'$$

$$E' \rightarrow + TE' / \in$$

$$T \rightarrow FT'$$

$$T' \rightarrow \times FT' / \in$$

$$F \rightarrow (E) / id$$

Now, the first and follow functions are as follows-

First Functions-

- First(E) = First(T) = First(F) = { (, id }
- First(E') = { + , ∈ }
- First(T) = First(F) = { (, id }
- First(T') = { × , ∈ }
- First(F) = { (, id }

Follow Functions-

- Follow(E) = { $,) }
- Follow(E') = Follow(E) = { $,) }
- Follow(T) = { First(E') – ∈ } ∪ Follow(E) ∪ Follow(E') = { + , $,) }
- Follow(T') = Follow(T) = { + , $,) }
- Follow(F) = { First(T') – ∈ } ∪ Follow(T) ∪ Follow(T') = { × , + , $,) }

- Follow(F) = { First(F) – ∈ } ∪ Follow(T) ∪ Follow(F) = { x , + , $,) }

Problem-06:

Calculate the first and follow functions for the given grammar-

$$S \rightarrow ACB / CbB / Ba$$

$$A \rightarrow da / BC$$

$$B \rightarrow g / \in$$

$$C \rightarrow h / \in$$

Solution-

The first and follow functions are as follows-

First Functions

- First(S) = { First(A) – ∈ } ∪ { First(C) – ∈ } ∪ First(B) ∪ First(b) ∪ { First(B) – ∈ } ∪ First(a) = { d , g , h , ∈ , b , a }
- First(A) = First(d) ∪ { First(B) – ∈ } ∪ First(C) = { d , g , h , ∈ }
- First(B) = { g , ∈ }
- First(C) = { h , ∈ }

Follow Functions-

- Follow(S) = { $ }
- Follow(A) = { First(C) – ∈ } ∪ { First(B) – ∈ } ∪ Follow(S) = { h , g , $ }
- Follow(B) = Follow(S) ∪ First(a) ∪ { First(C) – ∈ } ∪ Follow(A) = { $, a , h , g }
- Follow(C) = { First(B) – ∈ } ∪ Follow(S) ∪ First(b) ∪ Follow(A) = { g , $, b , h }

Left-Factoring

Left factoring is a process by which the grammar with common prefixes is transformed to make it useful for Top down parsers.

Example

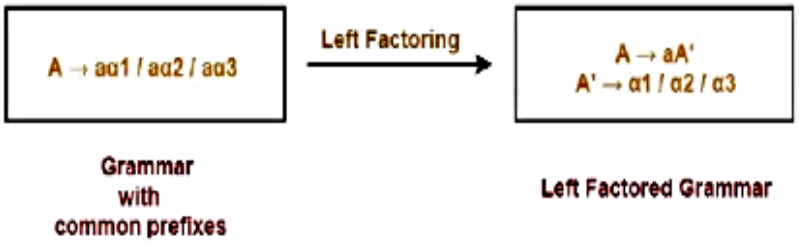
$A \rightarrow \alpha\beta1 / \alpha\beta2 / \alpha\beta3$

This kind of grammar creates a problematic situation for Top down parsers.

Top down parsers can not decide which production must be chosen to parse the string in hand.

To remove this confusion, we use left factoring.

Left-Factoring



Left-Factoring

Problem-1: Do left factoring in the following grammar-

A → aAB / aBc / aAc

A → aA'

A' → AB / Bc / Ac

A → aA'

A' → AD / Bc

D → B / c

This is a left factored grammar.

Left-Factoring

Problem-2: Do left factoring in the following grammar-

S → aSSbS / aSaSb / abb / b

S → aS' / b

S' → SSbS / SaSb / bb

S → aS' / b

S' → SA / bb

A → SbS / aSb

This is a left factored grammar.

Left-Recursion vs Left-Factoring

Left Recursion: A grammar is left recursive if it has a nonterminal A such that there is a derivation A -> Aα | β where α and β are sequences of terminals and nonterminals that do not start with A.

While designing a top down-parser, if the left recursion exist in the grammar then the parser falls in an infinite loop, here because A is trying to match A itself, which is not possible. We can eliminate the above left recursion by rewriting the offending production. As-

A -> βA'

A' -> αA' | epsilon

Left Factoring: Left factoring is required to eliminate non-determinism of a grammar. Suppose a grammar, S -> abS | aSb

Here, S is deriving the same terminal a in the production rule(two alternative choices for S), which follows non-determinism. We can rewrite the production to defer the decision of S as-

S -> aS'

S' -> bS | Sb

Thus, S' can be replaced for bS or Sb

LL (Parsing table

Now, after computing the First and Follow set for each *Non-Terminal symbol* we have to construct the Parsing table. In the table Rows will contain the Non-Terminals and the column will contain the Terminal Symbols.
All the **Null Productions** of the Grammars will go under the Follow elements and the remaining productions will lie under the elements of First set.
Now, let's understand with an example.

Example-1:
Consider the Grammar:

```
E --> TE'
E' --> +TE' | e
T --> FT'
T' --> *FT' | e
F --> id | (E)

**e denotes epsilon
```

Grammar	FIRST	FOLLOW
E –> TE'	{ id, (}	{ $,) }
E' –> +TE'/e	{ +, e }	{ $,) }
T –> FT'	{ id, (}	{ +, $,) }
T' –> *FT'/e	{ *, e }	{ +, $,) }
F –> id/(E)	{ id, (}	{ *, +, $,) }

Find their first and follow sets:

Find their first and follow sets:

Now, the LL(1) Parsing Table is:

	ID	+	*	(	)	$
E	E –> TE'			E –> TE'		
E'		E' –> +TE'			E' –> e	E' –> e
T	T –> FT'			T –> FT'		
T'		T' –> e	T' –> *FT		T' –> e	T' –> e
F	F –> id			F –> (E)		

As you can see that all the null productions are put under the follow set of that symbol and all the remaining productions are lie under the first of that symbol.

Note: Every grammar is not feasible for LL(1) Parsing table. It may be possible that one cell may contain more than one production.
Let's see with an example.

Example-2:
Consider the Grammar

```
S --> A | a
```

Download to

```
A --> a
```

Find their first and follow sets:

Grammar	FIRST	FOLLOW
S –> A/a	{ a }	{ $ }
A –>a	{ a }	{ $ }

Parsing Table:

	A	$
S	S –> A, S –> a	
A	A –> a	

Here, we can see that there are two productions into the same cell. Hence, this grammar is not feasible for LL(1) Parser.